



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

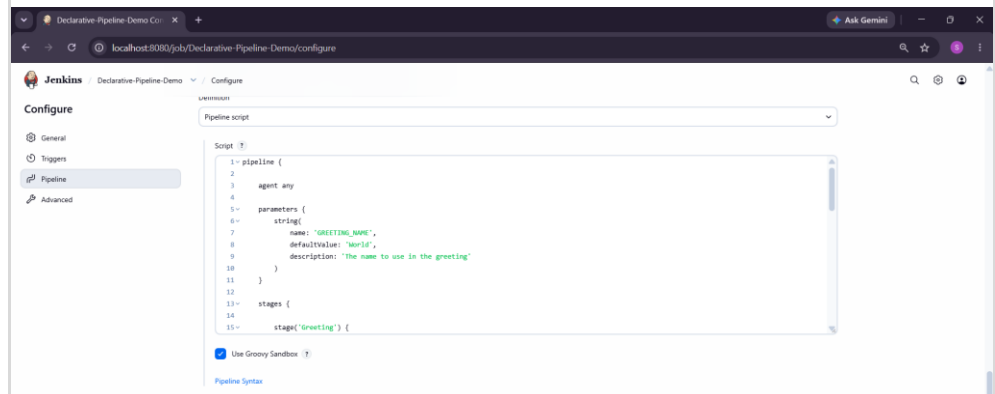
Department of Computer Science and Engineering
DevOps Lab - L3

Lab Assignment No: 14	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Date of Performance	30 th July, 2026
Title of Assignment	CI/CD: Jenkins Pipeline building Docker Image
Theory	A Jenkins Pipeline automates the Continuous Integration and Continuous Deployment (CI/CD) process by executing a series of predefined stages such as source code checkout, build, test, and deployment. In this experiment, a Docker image was built automatically using Jenkins after fetching the application source code from a GitHub repository. The docker build command created a versioned Docker image using the Jenkins build number as the image tag, ensuring unique image versions for every pipeline execution. This integration of Jenkins and Docker enables consistent, repeatable, and automated application builds, reducing manual effort and improving software delivery efficiency. Commands Used • docker build -t my-jenkins-docker . • docker run -d --name myjenkins --restart unless-stopped -u root -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock my-jenkins-docker • docker exec -it myjenkins docker --version • docker exec -it myjenkins rm -rf /var/jenkins_home/workspace/Declarative-Pipeline-Demo • git add . • git commit -m "Add Dockerfile for Jenkins pipeline"

- git push origin main
- docker images

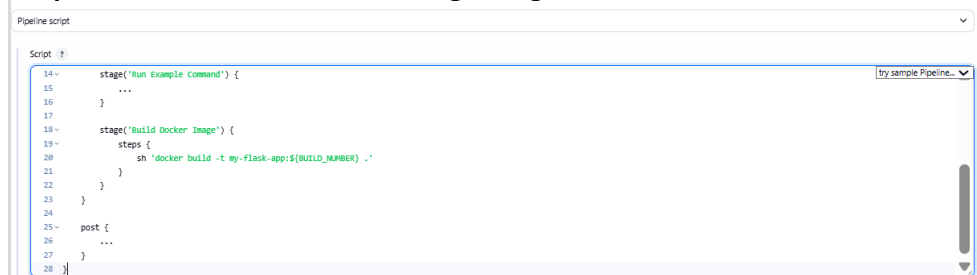
Implementation

Step 1: Access and Edit Your Pipeline Configuration



Accessed Jenkins Pipeline Configuration — Opened the existing Declarative Jenkins Pipeline and navigated to the Pipeline configuration section to modify the pipeline script.

Step 2: Add the Build Docker Image Stage



Added Docker Build Stage — Modified the existing Declarative Jenkins Pipeline by adding a new stage to build the Docker image using the docker build command with the Jenkins build number as the image tag.

Step 3: Run the Pipeline

```
C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo>echo --- Reading README ---
--- Reading README ---
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Declarative-Pipeline-Demo>type README.md
# Awesome Git Practice Project (Deature Update)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Executed Jenkins Pipeline — Successfully executed the Jenkins Declarative

Pipeline containing the Docker image build stage. The pipeline completed successfully, and the Docker image was built and tagged without errors.

Step 4: Verify the Docker Image

```
barry@SANVUKI MINGW64 ~/onedrive/desktop/jenkins-docker>docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
docker/desktop-cloud-provider-kind:v0.6.0	46d51b990e97	613MB	168MB	U
docker/desktop-containerd-registry-mirror:v0.0.4	1de142695bb0	52.9MB	16.9MB	U
envoyproxy/envoy:v1.36.7	29d778ba078e	253MB	63.9MB	U
hello-world:latest	c3cbe1cc1aa5	25.9kB	9.49kB	
jenkins/jenkins:lts-jdk17	cce1eeb79902	785MB	280MB	U
kindest/node:v1.36.1	3489c7674813	1.31GB	383MB	U
my-flask-app:8	2054b63c3d5b	200MB	48.8MB	
my-flask-app:latest	7375b8f52500	200MB	48.8MB	U
my-flask-app:v2	e0a6cd5edeae	200MB	48.8MB	U
my-jenkins-docker:latest	6d91d1c32234	1.4GB	442MB	U
redis:latest	234c902a2db4	208MB	56.3MB	

Verified Docker Image — Verified that the Docker image (my-flask-app:8) was successfully built by the Jenkins Pipeline and stored in the local Docker image repository using the docker images command.

Subject Faculty Name: Prof. Arijit Dutta